

Chaos-K

Chaos Engineering with Observability in Kubernetes

Autorzy: *(do uzupełnienia)*

Rok: 2026

Grupa: *(do uzupełnienia)*

Spis treści

1	Introduction	2
2	Theoretical background / technology stack	2
2.1	Chaos Engineering	2
2.2	Stack technologiczny	3
3	Case study concept description	3
3.1	Aplikacja	3
3.2	Observability	3
3.3	Wizualizacja	3
4	Case study high level architecture	3
5	Case study detailed architecture	4
6	Environment configuration description	4
6.1	Wymagania wstępne	4
6.2	Namespace'y w klastrze	4
7	Installation method	4
7.1	Instalacja minikube	4
7.2	Wdrożenie aplikacji Bookinfo	4
7.3	Instalacja Prometheusa i Grafany	5
7.4	Instalacja Krkn	5
8	Demo deployment steps	5
8.1	Configuration set-up	5
8.2	Data preparation	6
9	Demo description	6
9.1	Execution procedure	6
9.2	Results presentation	7
10	Summary — conclusions	7
11	References	7

1 Introduction

Projekt **Chaos-K** ma na celu zbudowanie zaawansowanego środowiska testowego dla aplikacji działających w Kubernetes, umożliwiającego przeprowadzanie kontrolowanych eksperymentów awaryjnych (*Chaos Engineering*) wraz z ich pogłębioną analizą za pomocą narzędzi obserwowalności (*observability*).

Inicjatywa skupia się na symulacji realistycznych scenariuszy awaryjnych — takich jak utrata węzła klastra, restarty podów czy degradacja sieci — a następnie analizie ich wpływu na wydajność, dostępność i stabilność systemu. Integracja z Prometheusem i Grafaną zapewnia wgląd w zachowanie systemu w czasie rzeczywistym.

Docelowo projekt zakłada integrację z LLM przez serwer MCP do generowania dynamicznych scenariuszy testowych oraz symulacji ruchu sieciowego — ten komponent na obecnym etapie **nie jest jeszcze zaimplementowany**.

Cele projektu

- Testowanie odporności systemu na awarie infrastruktury.
- Identyfikacja wąskich gardeł i problemów wydajnościowych.
- Integracja Chaos Engineering z Observability.
- Integracja Chaos-K z LLM + MCP Server (*planowane*).

2 Theoretical background / technology stack

2.1 Chaos Engineering

Chaos Engineering to dyscyplina polegająca na świadomym wprowadzaniu awarii do działającego systemu w celu oceny jego odporności i identyfikacji słabych punktów, zanim ujawnią się one w środowisku produkcyjnym. Krkn (Chaos-K) realizuje to podejście dla klastrów Kubernetes, oferując zestaw scenariuszy awarii:

- restarty i usuwanie podów (*pod kill*),
- przeciążenia CPU i pamięci (*hog scenarios*),
- awarie sieci (latencja, utrata pakietów),
- niedostępność węzłów.

Testy obejmują: definicję scenariusza, wstrzyknięcie awarii przez API Kubernetesa, obserwację reakcji systemu poprzez narzędzia monitorujące oraz analizę wyników w iteracyjnym cyklu poprawy odporności.

2.2 Stack technologiczny

Technologia	Zastosowanie
Kubernetes (minikube)	Lokalne uruchamianie i zarządzanie klastrem aplikacji testowej.
Docker Desktop / Hyper-V	Środowisko wirtualizacji wymagane przez minikube.
Krkn (Chaos Engineering)	Wstrzykiwanie awarii i testowanie odporności systemu.
Helm	Menedżer pakietów dla Kubernetes, używany do instalacji Prometheusa i Grafany.
Prometheus	Zbieranie metryk z klastra i aplikacji.
Grafana	Wizualizacja metryk i tworzenie dashboardów w czasie rzeczywistym.
Bookinfo (Istio)	Przykładowa aplikacja testowa do oceny odporności.
MCP + LLM	(planowane) symulacja ruchu sieciowego i generowanie dynamicznych scenariuszy testowych.

3 Case study concept description

Studium przypadku polega na testowaniu odporności aplikacji **Bookinfo** (przykładowej aplikacji z projektu Istio) uruchomionej w lokalnym klastrze Kubernetes (minikube). Podczas pracy aplikacji generowany jest ciągły ruch HTTP, a równolegle uruchamiane są scenariusze chaosu z poziomu narzędzia Krkn. Wyniki obserwowane są w Grafanie.

3.1 Aplikacja

Aplikacja **Bookinfo** — mikroserwisowa aplikacja prezentująca informacje o książce (productpage, details, reviews, ratings) — wdrażana w namespace **bookinfo** z oficjalnego manifestu Istio.

3.2 Observability

Prometheus (instalowany przez Helm chart **kube-prometheus-stack**) zbiera metryki z całego klastra Kubernetes oraz z podów aplikacji testowej.

3.3 Wizualizacja

Grafana (komponent stacka **kube-prometheus-stack**) wykorzystywana jest do prezentacji metryk w czasie rzeczywistym. Dashboardy importowane są po ID z repozytorium Grafany (np. ID 6417 — Kubernetes Cluster).

4 Case study high level architecture

Architektura wysokiego poziomu obejmuje cztery warstwy:

1. **Warstwa generowania ruchu / LLM + MCP** — symulacja ruchu sieciowego do aplikacji testowej. Obecnie: prosty load generator (**busybox** + **wget** w pętli). Docelowo: LLM + MCP Server.
2. **Chaos Layer** — Krkn zarządza wstrzykiwaniem awarii do klastra (usuwanie podów, hog CPU/RAM, problemy sieciowe).
3. **Observability Layer** — Prometheus zbiera metryki z klastra i aplikacji.
4. **Visualization Layer** — Grafana prezentuje dane i udostępnia konfigurowalne dashboardy.

5 Case study detailed architecture

(do uzupełnienia — szczegółowe diagramy, opis komunikacji między komponentami, namespace'y, service'y, port-forwardy)

6 Environment configuration description

6.1 Wymagania wstępne

- System: Windows 10/11 (testowane), Linux lub macOS.
- **Docker Desktop** lub **Hyper-V** — środowisko, w którym minikube uruchamia maszynę wirtualną klastra.
- **kubectl** — klient Kubernetesa.
- **Helm** — menedżer pakietów dla Kubernetes.
- **Python 3.11** — wymagany przez Krkn.
- **Git**.

6.2 Namespace'y w klastrze

- **bookinfo** — aplikacja testowa.
- **monitoring** — Prometheus + Grafana.

7 Installation method

7.1 Instalacja minikube

Minikube należy pobrać i zainstalować zgodnie z instrukcją z oficjalnej strony: minikube.sigs.k8s.io/docs/start. Przed pierwszym uruchomieniem należy upewnić się, że Docker Desktop (lub Hyper-V) jest uruchomiony.

```
minikube start
kubectl get nodes
```

7.2 Wdrożenie aplikacji Bookinfo

```
kubectl create namespace bookinfo
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.20/samples/bookinfo/platform/kube/bookinfo.yaml -n bookinfo
kubectl get pods -n bookinfo
```

Udostępnienie aplikacji na zewnątrz:

```
minikube service productpage -n bookinfo
```

7.3 Instalacja Prometheusa i Grafany

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring
```

Udostępnienie Grafany:

```
minikube service monitoring-grafana -n monitoring
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Pobranie hasła administratora Grafany (login: admin):

```
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d
```

Uwaga (Windows): polecenie `base64` nie jest dostępne domyślnie w CMD. Należy skorzystać z Git Bash, WSL lub PowerShella (`[System.Text.Encoding]::UTF8.GetString(`

`[System.Convert]::FromBase64String("..."))`).

Logowanie do Grafany: <http://localhost:3000>, następnie import dashboardu po ID pod adresem <http://localhost:3000/dashboard/import> (np. ID **6417**).

7.4 Instalacja Krkn

```
git clone https://github.com/redhat-chaos/krkn.git
cd krkn

py -3.11 -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

8 Demo deployment steps

8.1 Configuration set-up

Konfiguracja Krkn odbywa się przez plik `chaos.yaml`. Przykładowa zawartość uruchamiająca scenariusz CPU hog:

```
kraken:
  kubeconfig_path: "C:\\Users\\Szymon\\.kube\\config"
  distribution: kubernetes
  chaos_scenarios:
    - hog_scenarios:
        - scenarios/kube/cpu-hog.yml
  tunings:
    wait_duration: 60

performance_monitoring:
  prometheus_url: ""
  grafana_url: ""
```

```

elastic:
  enable_elastic: false
  elastic_url: ""

telemetry:
  enabled: false
  api_url: ""
  events_backup: false

cerberus:
  cerberus_enabled: false
  cerberus_url: ""

```

Scenariusz usuwania podów aplikacji Bookinfo — modyfikacja pliku scenariusza `regex-openshift-pod-kill`.

```

- id: kill-pods
  config:
    namespace_pattern: ^bookinfo$
    name_pattern: .*
    kill: 2
    krkn_pod_recovery_time: 120
    exclude_label: ""

```

8.2 Data preparation

Generowanie ruchu do aplikacji testowej — uruchomienie poda `busybox` w klastrze i wysyłanie zapytań w pętli:

```

kubectrl run loadgen --image=busybox -it -- /bin/sh

# wewn trz kontenera:
while true; do wget -q0- http://productpage:9080/productpage > /dev/
  null; sleep 0.2; done

```

9 Demo description

9.1 Execution procedure

Procedura uruchomienia demo:

1. Start klastra: `minikube start`.
2. Wdrożenie aplikacji Bookinfo (rozdział 7.2).
3. Wdrożenie stacku monitorującego (rozdział 7.3).
4. Import dashboardu w Grafanie (ID 6417).
5. Uruchomienie load generatora (rozdział 8.2).
6. Uruchomienie scenariusza chaosu z Krkn:

```
python run_kraken.py --config chaos.yaml
```

7. Obserwacja zachowania klastra w Grafanie podczas trwania scenariusza.

Przykładowe zapytanie PromQL pokazujące czas życia podów w namespace `bookinfo` (przydatne do śledzenia restartów wymuszonych przez Krkn):

```
time() - kube_pod_start_time{namespace="bookinfo"}
```

9.2 Results presentation

Promty użyte z modelami AI

Zrzuty ekranu z dashboardów Grafany

10 Summary — conclusions

11 References

- Repozytorium projektu Chaos-K: <https://github.com/Ciorolla/chaos-k>
- Krkn (Red Hat Chaos): <https://github.com/redhat-chaos/krkn>
- Minikube — dokumentacja: <https://minikube.sigs.k8s.io/docs/start/>
- Aplikacja testowa Bookinfo (Istio): <https://istio.io/latest/docs/examples/bookinfo/>
- Helm chart kube-prometheus-stack: <https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>
- Grafana: <https://grafana.com/>
- Prometheus: <https://prometheus.io/>
- Dashboard Grafana ID 6417 (Kubernetes Cluster): <https://grafana.com/grafana/dashboards/6417>